TESTER TOUT AU LONG DU CYCLE DE VIE DU DÉVELOPPEMENT LOGICIEL

Test des logiciels certification ISTQB

Sarra MEJRI:

- Docteur en informatique
- Sarra.mejri.isim@gmail.com

PLAN

1. Les cycles de vie de développement logiciel et les tests
2. Les niveaux de tests
3. Les types de tests
4. Les tests de maintenance

LES CYCLES DE VIE DE DÉVELOPPEMENT LOGICIEL ET LES TESTS

1

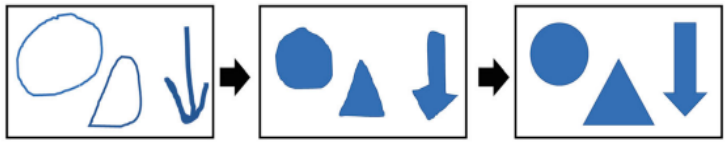
Les modèles de développement logiciel

LES MODÈLES DE DÉVELOPPEMENT LOGICIEL

Un modèle de cycle de vie de développement logiciel décrit :

- est **une représentation abstraite** de haut niveau du processus de développement logiciel.
- décrit **les activités** incluses dans le processus de développement logiciel et **les relations** entre elles.
- est utilisé pour déterminer comment le logiciel sera développé.

MODÈLE SÉQUENTIEL VS ITÉRATIF INCRÉMENTAL

Séquentiel	Itératif – Incrémental	
	Itératif	**Incrémental**
Il décrit le processus de développement logiciel comme un flux linéaire et séquentiel d'activités. **Toute phase du processus de développement devrait commencer lorsque la phase précédente est terminée.**	Il se déroule lorsque des **groupes de caractéristiques** sont spécifiés, conçus, développés et testés ensemble dans une série de **cycles**, souvent d'une durée fixe. Chaque itération délivre **un logiciel opérationnel** qui est un **sous-ensemble croissant de l'ensemble global des caractéristiques** jusqu'à ce que le logiciel final soit livré ou que le développement soit arrêté. 	La définition des exigences, la conception, le développement et le test d'un système **par morceaux**, ce qui signifie que **les fonctionnalités du logiciel augmentent de façon incrémentale.** La taille de ces incréments de fonctionnalités varie, certaines méthodes ayant des étapes plus grandes et d'autres plus petites. Les incréments de caractéristiques peuvent être aussi petits qu'une unique modification d'un écran d'interface utilisateur ou qu'une nouvelle option de requête.
<u>Exemples</u> **Modèle en cascade**, **modèle en V**	**<u>Exemples</u>** **Modèle en spirale de Boehm**, **prototypage**, **processus unifié**, **les processus agiles**	

MÉTHODE RIGIDE VS AGILE

Méthode Classique (ou rigide)	Méthode agile
Elle est caractérisée par sa rigidité et son manque de flexibilité face aux changements de besoins en cours de projet.	C'est une approche de gestion de projet qui se concentre sur des étapes **incrémentielles et itératives** pour achever les projets. Les parties incrémentales d'un projet sont réalisées dans des cycles de développement à court terme (**itérations ou sprints**). Cette approche privilégie **la livraison rapide**, **l'adaptation au changement** et **la collaboration** plutôt que la gestion descendante et le suivi d'un plan établi.
Exemples – Modèle en cascade – Modèle en V – Modèle en spirale de Boehm – Prototypage – Processus unifié	**Exemples** Scrum, Kanban, Extreme Programming (XP), Lean Software Development… Test-Driven Development (TDD), Acceptance Test-Driven Development (ATDD), Behavior-Driven Development (BDD), Domain-Driven Design (DDD), Feature-Driven Development (FDD).

Impact du cycle de vie du développement logiciel sur le test

MODÈLE DE CYCLE DE VIE DU DÉVELOPPEMENT LOGICIEL EN CONTEXTE

Mise en contexte?

- Les modèles de cycle de vie du développement logiciel doivent être sélectionnés et adaptés au contexte du projet et aux caractéristiques du produit.
- Un modèle de cycle de vie du développement logiciel approprié devrait être choisi et adapté en fonction de **l'objectif du projet**, du **type de produit développé**, des p**riorités métier** (p. ex., délai de mise sur le marché) **et des risques produit et projet identifiés…**
- Selon le contexte du projet, **il peut être nécessaire de combiner ou de réorganiser** les niveaux de test et/ou les activités de test.

CHOIX DU CYCLE DE DÉVELOPPEMENT LOGICIEL

Pour réussir, le test doit être adapté au cycle de vie du développement logiciel.

Le choix du cycle de vie du développement logiciel a une incidence sur:

- Le périmètre et le calendrier des activités de test (par exemple, les niveaux de test et les types de test).
- Le niveau de détail de la documentation des tests.
- Le choix des techniques de test et de l'approche de test.
- Le degré d'automatisation des tests.
- Le rôle et les responsabilités d'un testeur.

TESTER DANS LES MODÈLES DE DÉVELOPPEMENT SÉQUENTIEL

- Au cours des phases initiales, les testeurs participent généralement à la revue des exigences, à l'analyse de test et à la conception des tests.
- Le code exécutable est généralement créé dans les phases ultérieures, de sorte que les tests dynamiques ne peuvent généralement pas être effectués au début du cycle de vie du développement logiciel.

TESTER DANS :

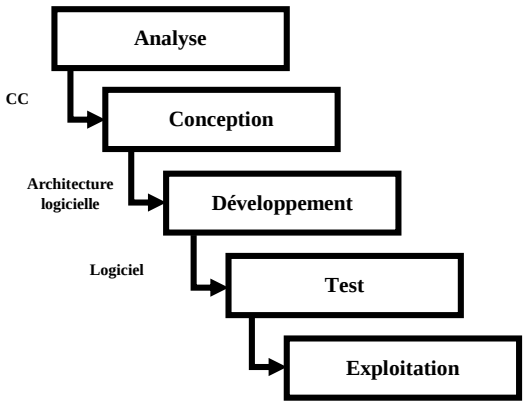
Modèle en cascade

Description :

Les activités de développement logiciel sont réalisées l'une après l'autre.

Exemples de projets :

- Dans les cas où les exigences sont bien connues et bien développées
- Dans les cas où les exigences sont très bien identifiées et où il y a peu ou pas de risque de changement.



Quand tester ?

Les activités de test ont lieu uniquement après que toutes les autres activités de développement ont été terminées.

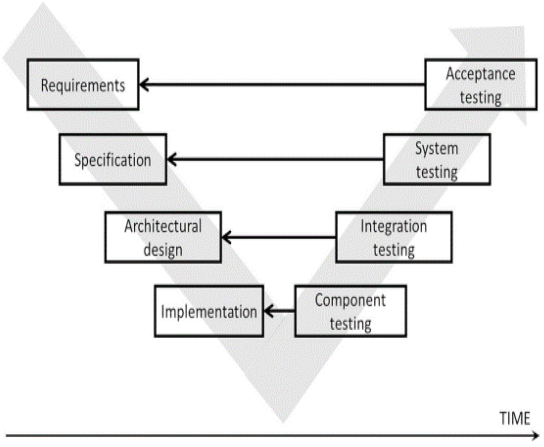
Modèle en V

Description :

C'est un modèle de développement modifié qui tente de résoudre les inconvénients du modèle en cascade, associés aux tests tardifs.

Projets :

- Dans les cas où les exigences sont relativement stables, où une validation approfondie est nécessaire à chaque étape du processus de développement
- Dans le cas où la traçabilité entre les exigences et les tests est cruciale pour assurer la qualité du produit final



Quand tester ?

- **Intégration du processus de test dans le processus de développement logiciel selon le principe du test précoce.**
- **Niveaux de test alignés avec chaque phase correspondante du développement, encourageant les tests précoces.**
- **Exécution séquentielle des tests à tous les niveaux, englobant la phase dynamique du test et les phases de développement.**

TESTER DANS CERTAINS MODÈLES DE DÉVELOPPEMENT INCRÉMENTAL ET ITÉRATIF

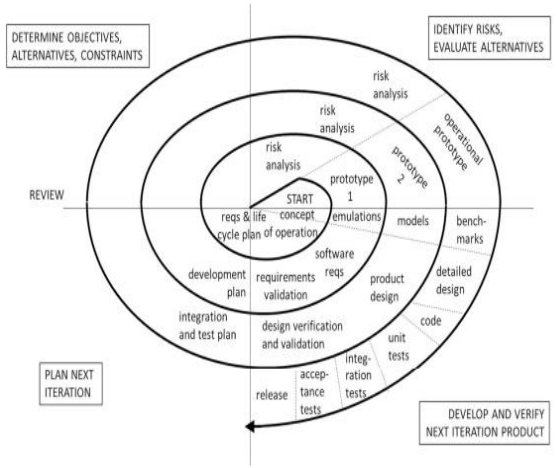
- En supposant que chaque itération fournit un prototype fonctionnant ou un incrément de produit.
- Cela implique qu'à chaque itération, des tests statiques et dynamiques peuvent être effectués à tous les niveaux de test.
- La livraison fréquente d'incréments nécessite un feedback rapide et des tests de régression poussés.

TESTER DANS:

Le modèle en spiral

Principe:

Le modèle en spirale est une approche dans laquelle des composants expérimentaux incrémentiels sont créés qui peuvent ensuite être largement reconstruits ou même abandonnés lors des étapes ultérieures du développement logiciel.



Quand tester :

Les composants ou systèmes utilisant les modèles susmentionnés intègrent souvent des niveaux de test redondants et superposés dans le cycle de développement logiciel.

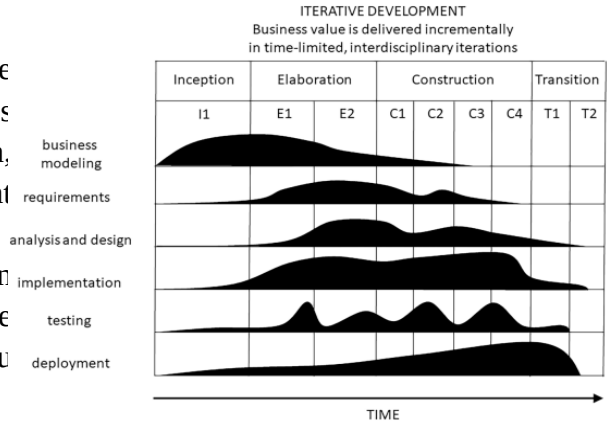
Idéalement, chaque fonctionnalité est testée à plusieurs niveaux, à mesure que l'on se rapproche de la version finale.

De nombreux modèles de ce type incluent le concept d'équipes auto-organisées, ce qui peut modifier la façon dont le travail est organisé en matière de test et la relation entre les testeurs et les développeurs.

Processus unifié

Principe : C'est un modèle itératif et incrémental :

- Incrémental : UP n'est pas un processus unique mais un cadre de processus au sein duquel des processus individuels sont définis, tels que la modélisation métier, les exigences, l'analyse et la conception, la mise en œuvre, les tests ou le déploiement. Ces processus peuvent être sélectionnés et adaptés aux besoins du projet.
- Itératif : Les itérations individuelles prennent généralement un temps relativement long , et les parties incrémentielles du système sont en conséquence importantes, couvrant, par exemple, deux ou trois groupes de fonctionnalités connexes.



Quand Tester :

Dans le modèle le processus de test est intégré à chaque phase du développement logiciel.

TESTER DANS LES MODÈLES AGILES

Le développement logiciel agile part du principe que des **changements** peuvent intervenir tout au long du projet.

- C'est pourquoi **les projets agiles** privilégient :
  - **une documentation légère des produits d'activités**
  - **une automatisation poussée des tests** pour faciliter les tests de régression.
- La plupart des tests manuels tendent à être effectués à l'aide de techniques de test basées sur l'expérience qui ne nécessitent pas d'analyse et de conception des tests préalables approfondies.

TESTER DANS:

Scrum

Scrum divise le développement logiciel en courtes itérations (sprints) de même durée (généralement 1 à 4 semaines), et les parties itératives du système sont en conséquence de petite taille.

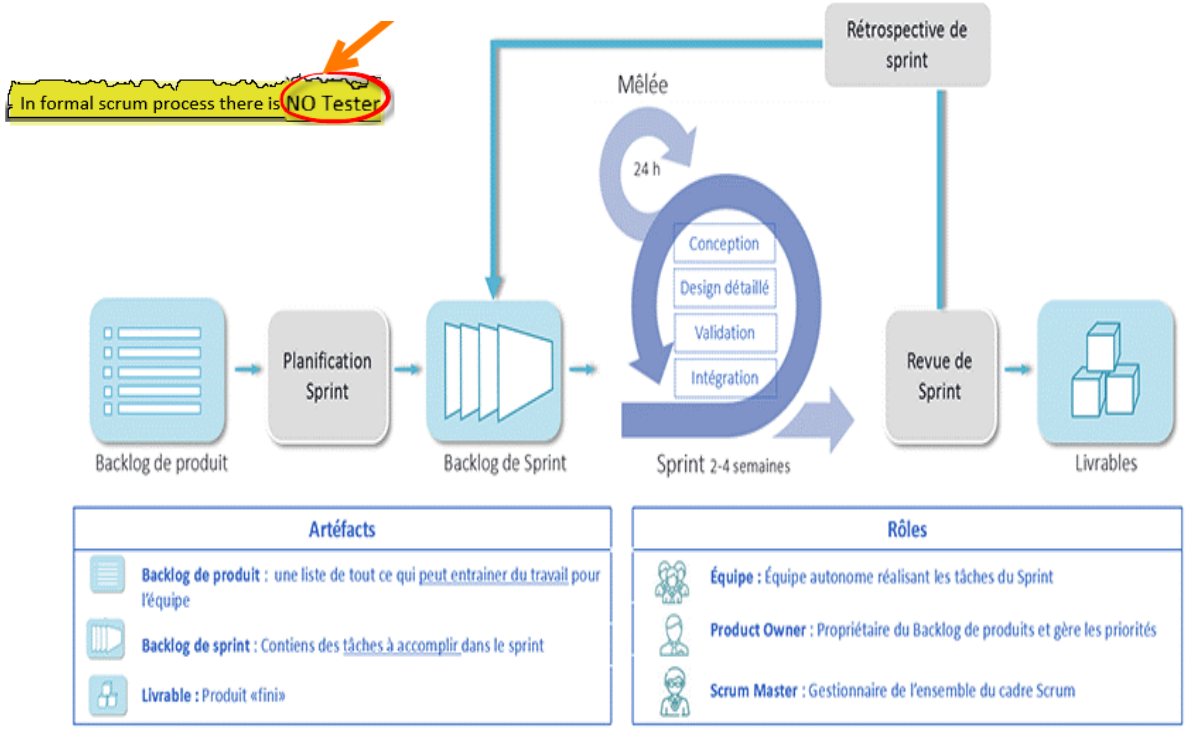
Un sprint est une itération qui produit un nouvel incrément (incrémental) et peut aussi enrichir l'incrément obtenu dans un sprint précédent (itératif).

Avant chaque itération, l'équipe sélectionne ("pousse") du backlog produit vers le backlog du sprint un ensemble prédéterminé de tâches à accomplir dans cette itération, et au sein d'un sprint, chaque membre de l'équipe commence à travailler sur la tâche suivante lorsque la précédente est terminée ("tirer").

Certains projets Scrum ont des équipes de test dédiées en fonction de la nature et de la complexité du projet

Les tests sont difficiles en raison des courtes itérations et des changements fréquents.

- **Mise en avant des tests exploratoires**
- **Mise en avant de l'automatisation des tests à grande échelle**



- **Équipe Scrum – rôles de Scrum**
  - **Product Owner – Scrum Master – Équipe de développement**
- **Événements**
  - **Cinq événements prescrits (à durée limitée) sont utilisés dans Scrum afin de créer de la régularité et de minimiser le temps passé en réunions**

    Scrum

    Planification de Sprint (*Sprint Planning*)
    Mêlée Quotidienne (*Daily Scrum*)
    Revue de Sprint (*Sprint Review*)
    Rétrospective de Sprint (*Sprint Retrospective*)
- **Artéfacts (Product Backlog, Sprint Backlog, Increment)**
  - **Ils représentent du travail ou de la valeur de diverses manières, de telle sorte qu'ils fournissent de la transparence et des opportunités pour l'inspection et l'adaptation**

LE TEST EN TANT QUE MOTEUR DU DEVELOPPEMENT LOGICIEL

TDD, ATDD & BDD

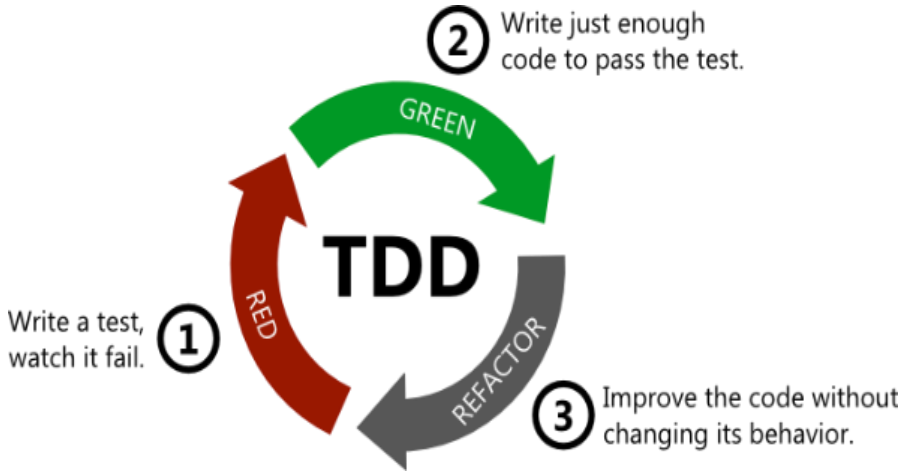
- TDD, ATDD et BDD sont des approches de développement similaires, dans lesquelles les tests sont définis comme un moyen d'orienter le développement.
- Chacune de ces approches implémente **le principe des tests précoces** et suit une approche **shift left**, puisque les tests sont définis avant que le code ne soit écrit. Elles soutiennent **un modèle de développement itératif**.
- Les tests peuvent persister en tant que **tests automatisés** afin de garantir la qualité du code lors des adaptations / refactorisations futures.

TDD

Le développement piloté par les tests - Test-Driven Development (TDD)

Le processus TDD, appelé RGR, se déroule comme suit :

- **R (Red): écrire un code de test et les faire échouer**
  →« Les tests sont écrits en premier »
- **G (Green) : écrire le code métier qui valide le test**
  → le code est écrit pour satisfaire les tests,
- **R (Refractor) : remaniement du code afin d'en améliorer la qualité**
  → les tests et le code sont refactorisés.



Les étapes ci-dessus sont répétées tant qu'il reste encore du code à écrire.

L'approche TDD :

- utilise **des cas de test automatisés** pour les composants afin de guider le développement du code → dirige le codage par le biais de cas de tests (au lieu d'une conception approfondie du logiciel).
- **est généralement effectué par un développeur.**
- utilise généralement **un framework de test** tel que **jUnit, pytest, nunit, phpunit ….** pour prendre en charge les tests automatisés.

En utilisant l'approche TDD, les développeurs :

- acquièrent une certaine indépendance vis-à-vis des tests.

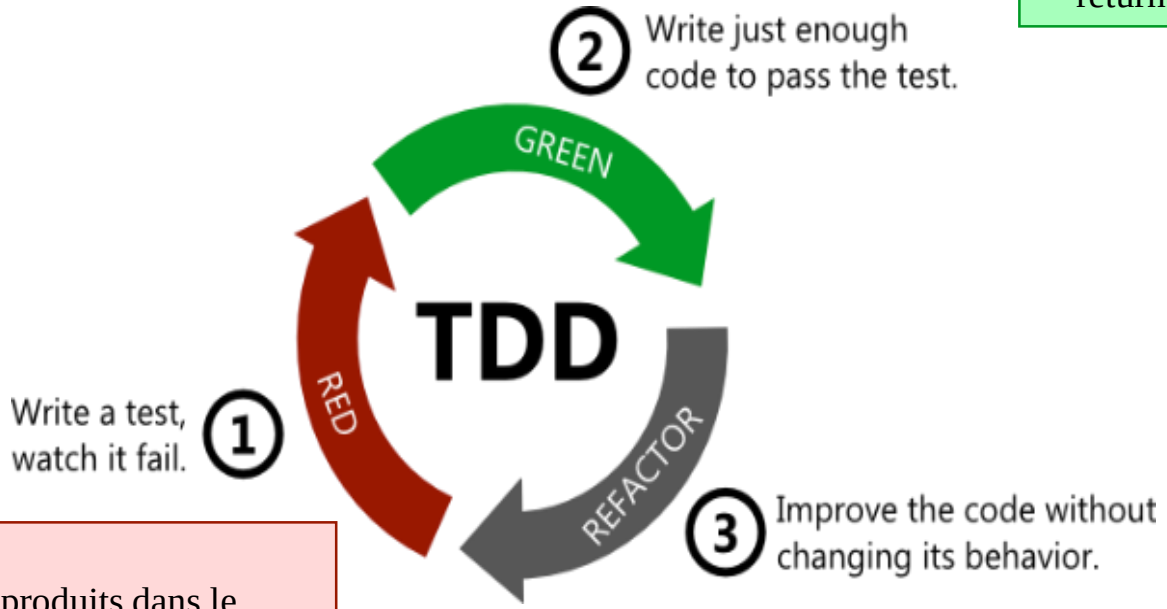
- n'ont pas de lien émotionnel avec le code, car ils ne l'ont pas encore créé.

TDD

exemple

Objectif: une fonctionnalité pour calculer le prix total des produits dans le panier d'achat

```
def calcul_prix_total(panier):
    return sum(panier)
```




```
def calcul_prix_total(panier):
    if not panier:
        return 0
    else:
        return sum(panier)
```

```
def test_calcul_prix_total():
    panier = [10, 20, 30]  # Les prix des produits dans le panier
    prix_total = calcul_prix_total(panier)
    assert prix_total == 60  # Le prix total devrait être la somme des prix des produits
```

BDD

Développement piloté par le comportement

Behavior-Driven Development (BDD)

- Le Behavior-Driven Development, ou BDD, est une méthode de développement logicielle dérivée du Test-Driven Development – TDD.

- En TDD, les tests peuvent même se référer à quelques lignes de code individuelles En BDD, développement piloté par **le comportement**, les tests se concentrent sur **le comportement attendu du système.**

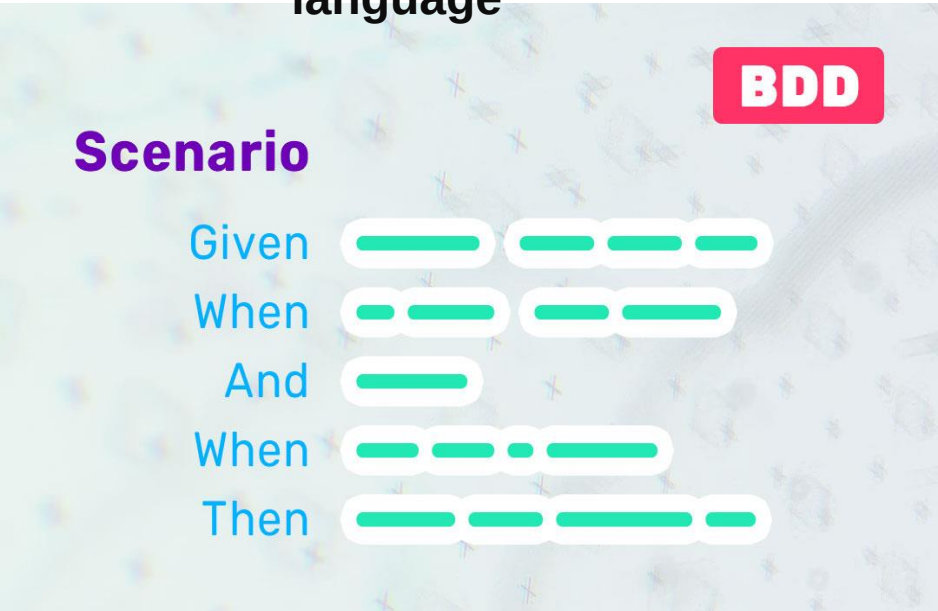
- BDD :

– utilise une approche collaborative pour **générer des critères d'acceptation en langage clair**, généralement dans le cadre d'une user story qui peut être comprise par toutes les parties prenantes. **Les critères d'acceptation sont généralement rédigés en utilisant des frameworks.** Le format typique pour les critères d'acceptation est étant Donné/lorsque/Alors (given, when, then).

– **peut être automatisé** : Le framework, basé sur la user story et les critères d'acceptation associés, génère automatiquement le code des cas de test et l'exécute.

– Exemples de framework : Cucumber, SpecFlow, Behave…

Gherkin language



•**Given - Étant donné :** Décrit l'état initial ou les préconditions requises pour que le scénario soit exécuté.
•**When - Quand :** Décrit l'action ou l'événement qui déclenche le comportement testé.
•**Then - Alors :** Décrit le résultat attendu ou le résultat du scénario.

BDD

exemple

écrivez le comportement de l'application pour saisir le nom d'utilisateur et le mot de passe en

Scénario: Vérification de la connexion
Etant donné que je suis sur la page de connexion
Quand j'entre le nom d'utilisateur "nom d'utilisateur"
Et que j'entre le mot de passe "Mot de passe"
Et je clique sur le bouton "Connexion"
Ensuite, je peux me connecter avec succès.

1

Écrire le script de test automatisé.

```java
@Steps
HomePage hp;
    0 références
    @Given("^I am on the login page $")
    0 références
    public void i_am_on_the_login_page()
{
    loginPage.gotoLoginPage();
}

0 références
@When("^I enter \"([^\"]*)\" username$")
    0 références
    public void i_enter_something_username(String username)
{
    loginPage.enterUserName(username);
}

0 références
@When("^I enter \"([^\"]*)\" password$")
    0 références
    public void i_enter_something_password(String password)
{
    loginPage.enterPassword(password);
}

0 références
@When("^I click on the \"([^\"]*)\" button$")
    0 références
    public void i_click_on_the_submit_button(String strArg1)
{
    hp = loginPage.submit();
}

0 références
@Then("^I am able to login successfully\.$")
    0 références
    public void i_am_able_to_login_successfully()
{
    Assert.assertNotNull(hp);
}
}
```

2

implémenter le code fonctionnel

```java
0 références
public class LoginPage
{
    String username = "";
    String password = "";
    //store username
    0 références
    public void enterUserName(String username)
    {
        this.username = username;
    }
    //store password
    0 références
    public void enterPassword(String password)
    {
        this.password = password;
    }
    //match username and passowrd in db and return home page
    0 références
    public HomePage submit()
    {
        if (username.existsInDB())
        {
            String dbPassword = getPasswordFromDB(username);
            if (dbPassword.equals(password){
                Return new HomePage();
            }
        }
    }
```

3

Remanier le code.

```java
//match username and passowrd in db and return home page
0 références
public HomePage submit()
{
    if (username.existsInDB())
    {
        String dbPassword = getPasswordFromDB(username);
        if (dbPassword.equals(password){
            Return new HomePage();
        }
        else
        {
            System.out.println("Please provide correct password");
            return;
        }
    }
    else
    {
        System.out.println("Please provide correct username");
    }
```

ATDD

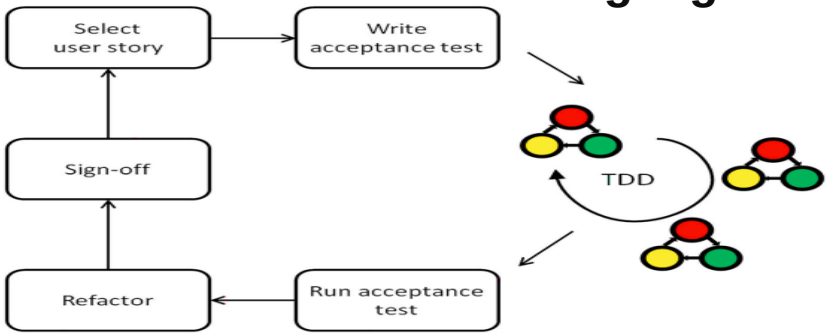
Le développement piloté par les tests d'acceptation - Acceptance Test-Driven Development (ATDD)

- **L'ATDD** suit le même procédé que celui défini pour le TDD et le BDD, mais avec **des cas de test automatisés qui sont au niveau des tests d'acceptation.**
- **Ces cas de test d'acceptation sont dérivés des critères d'acceptation qui sont générés conjointement par les développeurs, les testeurs et les représentants métier.**
- Les critères d'acceptation sont rédigés du point de vue de l'utilisateur et sont généralement dans un format facile à comprendre (par exemple, Etant donné/Quand/Alors, connu de l'approche BDD).

Gherkin language

Le processus de mise en œuvre de l'ATDD est le suivant :

- Sélectionner une user story
- Rédiger un test d'acceptation
- Implémenter une user story (codage)
- Exécuter un test d'acceptation
- Refactoriser le code, si nécessaire
- Obtenir l'approbation de la user story (validation)



L'ATDD utilise généralement un framework de test d'acceptation automatisé (tel que FitNesse) pour prendre en charge les tests d'acceptation automatisés.

ATDD

exemples

User Story 1:
Titre: Rechercher des produits par nom
En tant que utilisateur,
Je veux rechercher des produits par leur nom,
Afin que je puisse rapidement trouver les produits que je veux acheter.

Critères d'acceptation :
1.Étant donné que je suis sur la page de recherche de produits,
Quand je saisis "iPhone" dans la barre de recherche,
Alors je devrais voir une liste de produits contenant "iPhone" dans leur nom.

User Story 2:
Titre: Gestion des termes de recherche invalides
En tant que utilisateur,
Je veux être informé lorsque je saisis un terme de recherche invalide,
Afin que je puisse comprendre pourquoi je ne vois pas de résultats de recherche.

Critères d'acceptation :
Étant donné que je suis sur la page de recherche de produits,
Quand je saisis un terme de recherche invalide,
Alors je devrais voir un message indiquant qu'aucun produit n'a été trouvé.

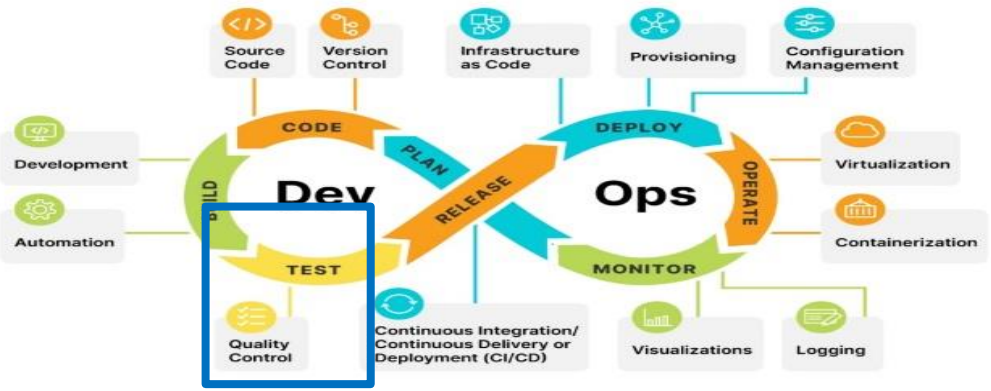
DEVOPS ET TEST

DEVOPS

Developement and Operations

- DevOps est une approche organisationnelle visant à créer une synergie en amenant le **développement** (y compris le test) et **l'exploitation** à travailler ensemble pour atteindre un ensemble d'objectifs communs.

- DevOps favorise :
  - **l'autonomie des équipes**
  - **le retour d'information rapide**
  - **les chaînes d'outils intégrées**
  - **les pratiques techniques** telles que l'**Intégration Continue (CI)** et **la livraison continue (CD).**



Cela permet aux équipes de construire, de tester et de livrer plus rapidement un code de haute qualité par le biais d'un pipeline de livraison DevOps.

PRINCIPES DE DEVOPS

- **Modèle itératif/incrémental**
- **Principes agiles:** autonomie de l'équipe, retour d'information rapide, et chaînes d'outils intégrées
- **CI:** Intégration continue. Intégrer fréquemment les changements de code dans un référentiel partagé et exécuter automatiquement des tests pour détecter les problèmes d'intégration tôt.
- **CD:** Livraison continue de logiciels ou Déploiement continu de logiciels. S'assurer que les changements de code sont toujours dans un état déployable, permettant des livraisons rapides et fiables.
- **Automatisation:** Automatiser les tâches répétitives, telles que la construction du code, les tests et le déploiement, pour augmenter l'efficacité et réduire les erreurs humaines. Automatiser les activités manuelles chaque fois que possible.
- **Infrastructure en tant que code:** Gérer les ressources d'infrastructure (par exemple, serveurs, réseaux et bases de données) en utilisant du code et des outils d'automatisation pour améliorer la cohérence et la scalabilité. Cela supporte la configuration automatisée des environnements de test et de production pour prendre en charge la livraison continue et le déploiement continu.

TESTER DANS DEVOPS

Automatisation des tests

- **Autant de tests que possible doivent être automatisés**, car sinon les tests ralentissent la livraison du code et permettent potentiellement aux défauts de se glisser en production.
- Ces **tests automatisés** devraient fournir un retour d'information rapide sur les défauts trouvés, ainsi que soutenir l'intégration continue, la livraison continue et le déploiement continu.

AVANTAGES DE DEVOPS

- Fournir un feedback rapide sur la qualité du code et sur l'impact éventuel des modifications sur le code existant.
- Favoriser une approche shift left du test en encourageant les développeurs à soumettre un code de haute qualité accompagné de tests de composants et d'une analyse statique.
- Favoriser l'automatisation de processus tels que CI/CD qui facilitent la mise en place d'environnements de test stables.
- Augmenter la vue sur les caractéristiques qualité non fonctionnelles (par exemple, la performance, la fiabilité).
- Réduire le besoin de tests manuels répétitifs grâce à l’automatisation par le biais d'un pipeline de livraison.
- Minimiser le risque de régression grâce à l'ampleur et à la portée des tests de régression automatisés.

DÉFIS

DevOps n'est pas sans risques et défis, lesquels incluent notamment que :

- Le pipeline de livraison DevOps doit être défini et établi.
- Les outils de CI/CD doivent être introduits et maintenus
- L'automatisation des tests nécessite des exigences supplémentaires et peut être difficile à mettre en place et à maintenir
- Bien que DevOps s'accompagne d'un niveau élevé de tests automatisés, des tests manuels – en particulier du point de vue de l'utilisateur - seront toujours nécessaires.

APPROCHE SHIFT LEFT

APPROCHE SHIFT LEFT

- Le principe du test précoce est parfois appelé shift left parce qu'il s'agit d'une approche dans laquelle le test est effectué plus tôt dans le cycle de vie du développement logiciel.
- Le principe du shift left suggère normalement que les tests doivent être effectués plus tôt (par exemple, sans attendre que le code soit implémenté ou que les composants soient intégrés), mais il ne signifie pas que les tests doivent être négligés dans les phases ultérieures du cycle de vie du développement logiciel
- Une approche shift left peut entraîner des formations, des efforts et/ou des coûts supplémentaires au début du processus, mais devrait permettre d'économiser des efforts et/ou des coûts à un stade ultérieur du processus.
- Pour l'approche shift left, il est important que les parties prenantes soient convaincues et adhèrent à ce concept.

BONNES PRATIQUES

Il existe quelques bonnes pratiques qui illustrent comment réaliser un "shift left" dans le test :

- Examiner la spécification du point de vue des tests. Ces activités de revue des spécifications permettent souvent de trouver des défauts potentiels, tels que des ambiguïtés, des incomplétudes et des incohérences.
- Rédiger des cas de test avant l'écriture du code et faire exécuter le code dans un harnais de test pendant l'implémentation du code.
- Utiliser l’Intégration Continue et, mieux encore, le Développement Continu, permettant un feedback rapide et des tests de composants automatisés pour accompagner le code source lorsqu'il est déposé dans le référentiel de code.
- Clôturer les tests statiques du code source avant les tests dynamiques, ou dans le cadre d’un processus automatisé
- Effectuer des tests non fonctionnels en commençant par le niveau de test de composants, dans la mesure du possible. Il s'agit d'une forme de shift left, car ces types de tests non fonctionnels ont tendance à être réalisés plus tard dans le cycle de vie du développement logiciel, lorsqu’un système complet et un environnement de test représentatif sont disponibles.

Retrospectives et améliorations des test

LES RÉTROSPECTIVES

- Les rétrospectives (également appelées "réunions post-projet" et rétrospectives de projet) sont souvent organisées à la fin d'un projet ou d'une itération, lors d'une étape de livraison, ou peuvent être organisées en cas de besoin.
- Le calendrier et l'organisation des rétrospectives dépendent du modèle de cycle de vie du développement logiciel suivi. Lors de ces réunions, les participants (non seulement les testeurs, mais aussi, par exemple, les développeurs, les architectes, le Product Owner, les analystes métier) discutent des points suivants:
- Qu'est-ce qui a été un succès et qui devrait être conservé ?
- Qu'est-ce qui n'a pas fonctionné et qui pourrait être amélioré ?
- Comment intégrer les améliorations et conserver les succès à l'avenir ?

LES RÉTROSPECTIVES

- Les résultats doivent être enregistrés et font normalement partie du rapport de clôture des tests. Les rétrospectives sont essentielles pour la mise en œuvre réussie de l'amélioration continue et il est important que toutes les améliorations recommandées soient suivies.
- Les avantages typiques pour les tests sont les suivants:
  - Augmentation de l 'efficacité et de l 'efficience des tests (par exemple, en mettant en œuvre des suggestions d'amélioration du processus de test).
  - Augmentation de la qualité du testware (par exemple, en examinant conjointement les processus de test).
  - Consolidation de l'équipe et apprentissage (par exemple, en raison de la possibilité de soulever des problèmes et de proposer des points d'amélioration).
  - Amélioration de la qualité de la base de test (par exemple, car les lacunes dans l'étendue et la qualité des exigences peuvent être abordées et résolues).
  - Amélioration de la coopération entre le développement et le test (par exemple, parce que la collaboration est régulièrement examinée et optimisée).

NIVEAUX DE TEST

2

LES NIVEAUX DE TEST

- Les niveaux de test sont des groupes d'activités de test qui sont organisés et gérés ensemble.
- Chaque niveau de test est une instance du processus de test, exécutée en relation avec un logiciel à un stade de développement donné, depuis les composants individuels jusqu'aux systèmes complets ou, le cas échéant, aux systèmes de systèmes.
- Les niveaux de test se distinguent par la liste, non exhaustive, suivante d'attributs, afin d'éviter le chevauchement des activités de test:
  - ✓ Objet de test.
  - ✓ Objectifs du test.
  - ✓ Base de test.
  - ✓ Défauts et défaillances.
  - ✓ Approche et responsabilités.

LE NIVEAUX DE TEST

Test de composants

- sont connus sous le nom de tests unitaires
- se concentrent sur les tests de composants isolés.
- nécessitent souvent un support spécifique, tel que des harnais de tests ou des frameworks de tests unitaires.
- sont normalement effectués par les développeurs dans leur environnement de test.

```
program pgcd ;
uses wincrt ;
var    a , b : integer ;
begin
  write('Donner a= ') ;
  readln(a) ;
  write('Donner b= ') ;
  readln(b) ;
     while a<>b do
        begin
        if a>b then a:=a-b
                 else b:=b-a ;
        end ;

writeln('Le pgcd est ', a) ;
end.
```

```
1  public class Point {
2      private int x;
3      private int y;

5      public int gety() {return y;}

7      public Point setx(int i) {
8          x = i;
9          return this;}
10 }
```

Test de composants

LE NIVEAUX DE TEST

Test d'intégration de composants

- connus sous le nom de tests d'intégration unitaires.
- se concentrent sur le test des interfaces et des interactions entre les composants.
- dépendent fortement des approches de la stratégie d'intégration:
  - Ascendante: les composants les plus basiques et les plus fondamentaux sont intégrés en premier.
  - descendante: commence par l'intégration des composants de niveau supérieur du système.
  - big-ban : consiste à intégrer tous les composants du système simultanément, souvent à la fin du cycle de développement.
- se concentrent sur les interactions et les interfaces entre composants intégrés.
- sont effectués après les tests de composants et sont généralement automatisés.

Tests de d'intégration de composants

Test de composants

LE NIVEAUX DE TEST

Tests système

- se concentrent sur le comportement global et les capacités d'un système ou d'un produit entier, et comprennent souvent des tests fonctionnels, des tâches de bout en bout et des tests non fonctionnels de caractéristiques-qualité. Pour certaines caractéristiques-qualité non fonctionnelles, il est préférable de les tester sur un système complet dans un environnement de test représentatif (par exemple, utilisabilité). Il est également possible d'utiliser des simulateurs de sous-systèmes. Le test système peut être effectué par une équipe de test indépendante et est lié aux spécifications du système.

Tests système

Tests de d'intégration de composants

Test de composants

LE NIVEAUX DE TEST

Tests d'intégration système

- se concentrent sur les interactions et les interfaces entre systèmes, progiciels et micro-services.
- peuvent également couvrir les interactions avec des entités externes (p. ex. services Web) et les interfaces fournies par celles-ci.
- peuvent être effectués après les tests systèmes ou en parallèle d'activités de tests systèmes en cours.

Tests de d'intégration du système

Tests système

Tests de d'intégration de composants

Test de composants

LE NIVEAUX DE TEST

Tests d'acceptation

- sont axés sur la validation et la démonstration de l'aptitude au déploiement, ce qui signifie que le système répond aux besoins métier de l'utilisateur.
- Idéalement, les tests d'acceptation devraient être effectués par les utilisateurs prévus.
- Les principales formes de tests d'acceptation sont :

1.TA des utilisateurs

TA opérationnel

TA contractuel et réglementaire

tests alpha

tests bêta

Tests d'acceptation

Tests de d'intégration du système

Tests système

Tests de d'intégration de composants

Test de composants

LE NIVEAUX DE TEST

Tests d'acceptation

TA utilisateur

- Validation par les utilisateurs dans un environnement opérationnel réel ou simulé
- Réponse aux besoins des utilisateurs

TA opérationnels

- par le service d'exploitation ou le service d'administration du système sont généralement effectués dans un environnement de production (simulé)
- Les opérateurs et les administrateurs peuvent maintenir le système

TA contractuels et réglementaires

- TA contractuelle: en fonction des critères d'acceptation d'un contrat pour la production de logiciels sur mesure.
- TA réglementaire : par rapport à tous les règlements qui doivent être respectés, comme les règlements gouvernementaux, légaux ou de sécurité

Tests alpha

- sur le site de l'organisation qui développe (dans les locaux de l'organisation)
- non pas par l'équipe de développement, mais par des clients potentiels ou existants, et/ou des opérateurs ou une équipe de test indépendante.

Tests bêta

- par des clients potentiels ou existants, et/ou des opérateurs
- sur leurs propres sites.
- après le test alpha, ou sans qu'aucun test alpha précédent n'ait eu lieu.

Tests d'acceptation

Tests de d'intégration du système

Tests système

Tests de d'intégration de composants

Test de composants

LES TYPES DE TESTS

LES TYPES DE TESTS

- Un type de test est un groupe d'activités de test visant à tester des caractéristiques spécifiques d'un système logiciel ou d'une partie d'un système, sur la base d'objectifs de test spécifiques .
- Les différents types sont:

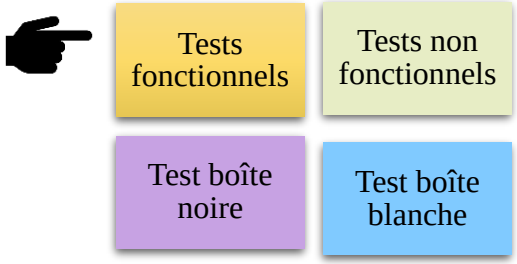
Tests fonctionnels

Tests non fonctionnels

Test boîte noire

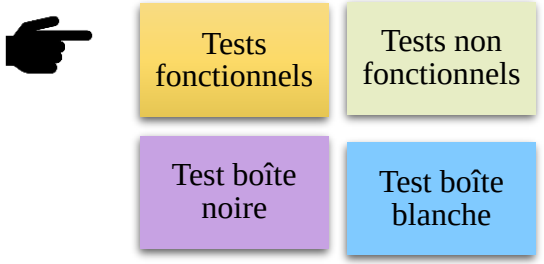
Test boîte blanche

LES TESTS FONCTIONNELS



- Les tests fonctionnels d'un système impliquent des tests qui évaluent les **fonctionnalités** que le système devrait réaliser.
- **Est-ce que le logiciel est conforme à la spécification ?**
- Les fonctionnalités sont « ce que » le système doit faire.
- L'objectif principal du test fonctionnel est de vérifier la complétude fonctionnelle, l'exactitude fonctionnelle et l'adéquation fonctionnelle.
- Les exigences fonctionnelles peuvent :
  - ✓ être décrites dans **des produits d'activités** tels que les spécifications des exigences métier, les épics, les User Stories, les cas d'utilisation ou les spécifications fonctionnelles,
  - ✓ ne pas être documentées.

EXEMPLES DE TESTS FONCTIONNELS



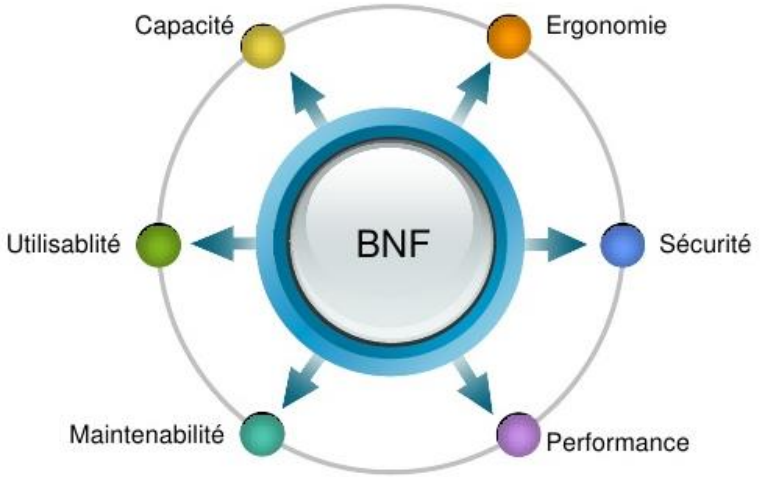
Pour une application bancaire

- **Pour les tests de composants**, les tests sont conçus en fonction de la façon dont un composant doit calculer les intérêts composés.
- **Pour les tests d'intégration de composants,** les tests sont conçus en fonction de la façon dont les informations saisies au niveau de l'interface utilisateur sont transmises à la couche métier.
- **Pour les tests système**, les tests sont conçus en fonction de la façon dont les titulaires de comptes peuvent demander une ligne de crédit sur leurs comptes chèques.
- **Pour les tests d'intégration au niveau système**, les tests sont conçus en fonction de la façon dont le système utilise un micro-service externe pour vérifier le taux de crédit d'un titulaire de compte.
- **Pour les tests d'acceptation**, les tests sont conçus en fonction de la façon dont le banquier gère l'approbation ou le refus d'une demande de crédit

Les tests fonctionnels devraient être effectués **à tous les niveaux de test** (par exemple, les tests de composants peuvent être basés sur une spécification de composant), bien que la focalisation soit différente à chaque niveau.

LES TESTS NON FONCTIONNELS

- Les tests non-fonctionnels d'un système évaluent les caractéristiques des systèmes et des logiciels comme l'utilisabilité, la performance ou la sécurité.
- Les tests non fonctionnels sont les tests de "comment" le système se comporte évalue les attributs autres que les caractéristiques fonctionnelles d'un composant ou d'un système. Le test non fonctionnel est le test de **"comment le système se comporte".**

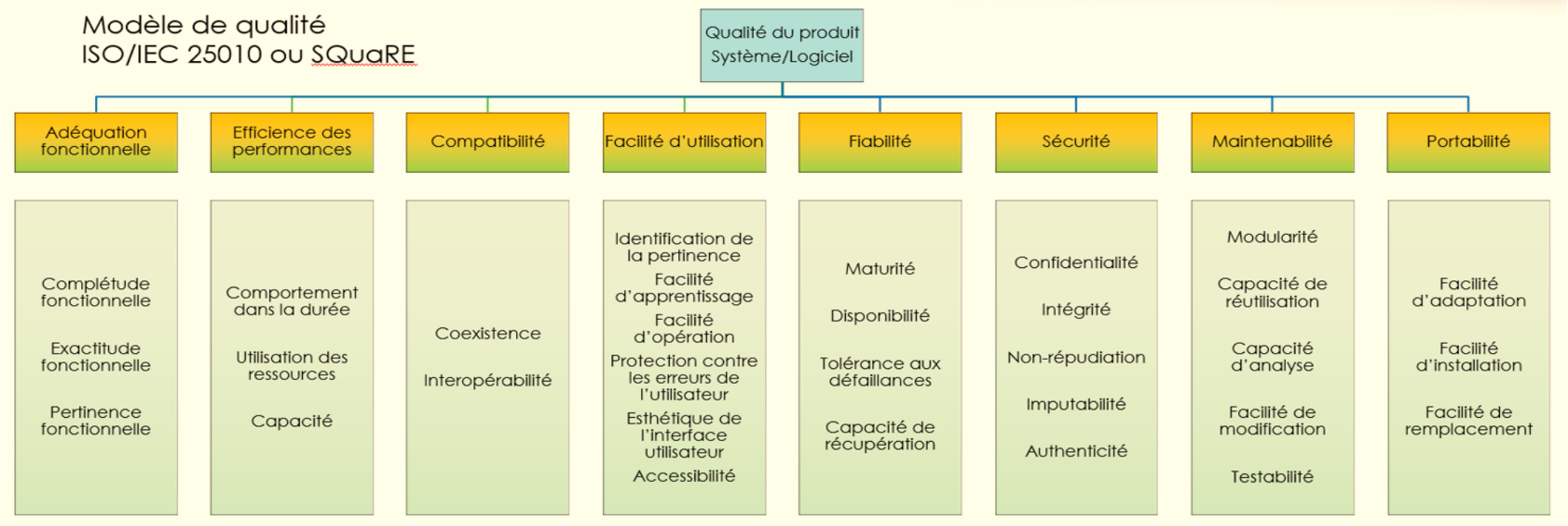


LES TESTS NON FONCTIONNELS

ISO/IEC 25010

Tests fonctionnels | Tests non fonctionnels | Test boîte noire | Test boîte blanche

- L'objectif principal du test non fonctionnel est de vérifier les caractéristiques-qualité non fonctionnelles du logiciel. Le standard ISO/IEC 25010 fournit la classification suivante des caractéristiques qualité non fonctionnelles des logiciels : efficacité de la performance, Compatibilité, Utilisabilité, Fiabilité, Sécurité, Maintenabilité, Portabilité…



LES TESTS NON FONCTIONNELS

Exemples de métriques

Tests fonctionnels | Tests non fonctionnels

Test boîte noire | Test boîte blanche

Caractéristiques	Métriques	Modèle	description
Performance	Temps de réponse	M = T1 - T2	T1 = temps de réalisation de la tâche T2 = temps d'achèvement de l'émission d'une demande de tâche
	Débit (quantité de travail effectuée)	M=A	A = nombre de tâches terminées dans une unité de temps fixe
Utilisabilité	Facilité d'apprentissage	M = T	T = temps d'apprentissage
	Disponibilité de l'aide	M = A/B	A = le nombre de tâches pour lesquelles il existe un sujet d'aide valide B = nombre total de tâches testées
	Attrait de l'interaction	questionnaire	Mesures de la satisfaction utilisateur évaluées par questionnaire après l'utilisation du logiciel.

EXEMPLES DE TESTS NON-FONCTIONNELS

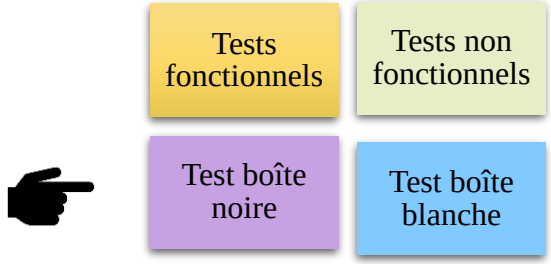
pour une application bancaire

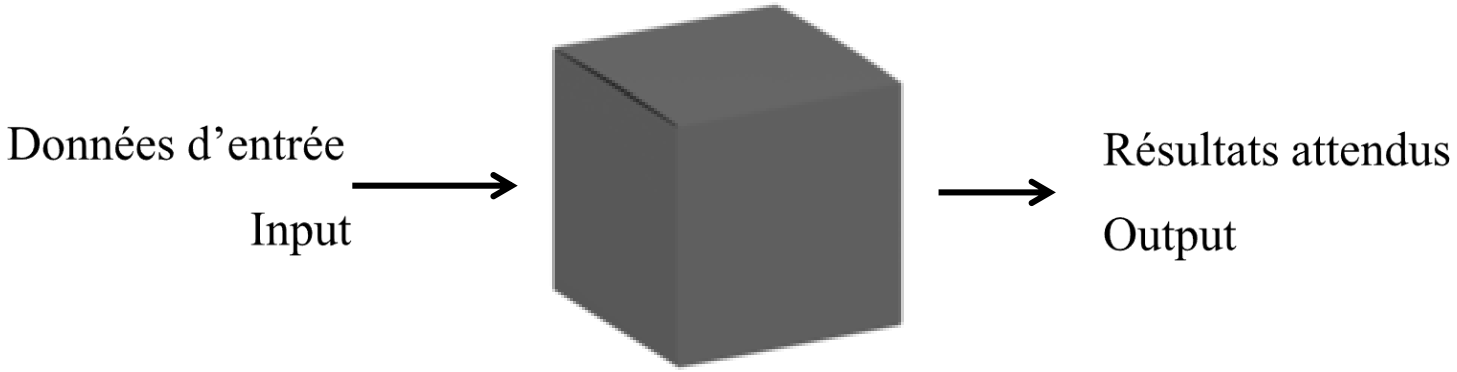
Tests fonctionnels | Tests non fonctionnels | Test boîte noire | Test boîte blanche

- **Pour les tests de composants**, les tests de performance sont conçus pour évaluer le nombre de cycles CPU nécessaires pour effectuer un calcul d'intérêt total complexe.
- **Pour les tests d'intégration de composants**, les tests de sécurité sont conçus pour les vulnérabilités de débordement de la mémoire tampon dues aux données transmises de l'interface utilisateur à la couche métier.
- **Pour les tests système**, les tests de portabilité sont conçus pour vérifier si la couche de présentation fonctionne sur tous les navigateurs et appareils mobiles supportés.
- **Pour les tests d'intégration de système**, les tests de fiabilité sont conçus pour évaluer la robustesse du système lorsque le micro-service de calcul du taux de crédit ne répond pas.
- **Pour les tests d'acceptation**, les tests d'utilisabilité sont conçus pour évaluer l'accessibilité de l'interface de traitement du crédit du banquier pour les personnes handicapées.

Les tests non fonctionnels devraient être effectués **à tous les niveaux de test**.

LES TESTS BOÎTE NOIRE





Les système sont vus comme une boîte noire !!!

Le test boîte noire est basé sur les spécifications et dérive les tests de la documentation externe à l'objet de test. L'objectif principal du test boîte noire est de vérifier le comportement du système par rapport à ses spécifications.

TECHNIQUES DE TESTS BOÎTE NOIRE

Tests fonctionnels

Tests non fonctionnels

Test boîte noire

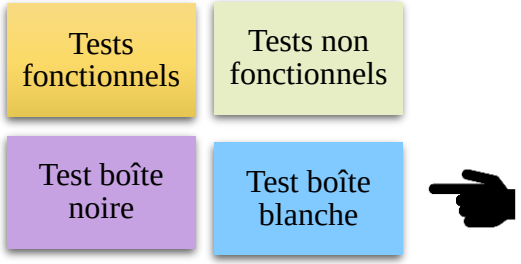
Test boîte blanche

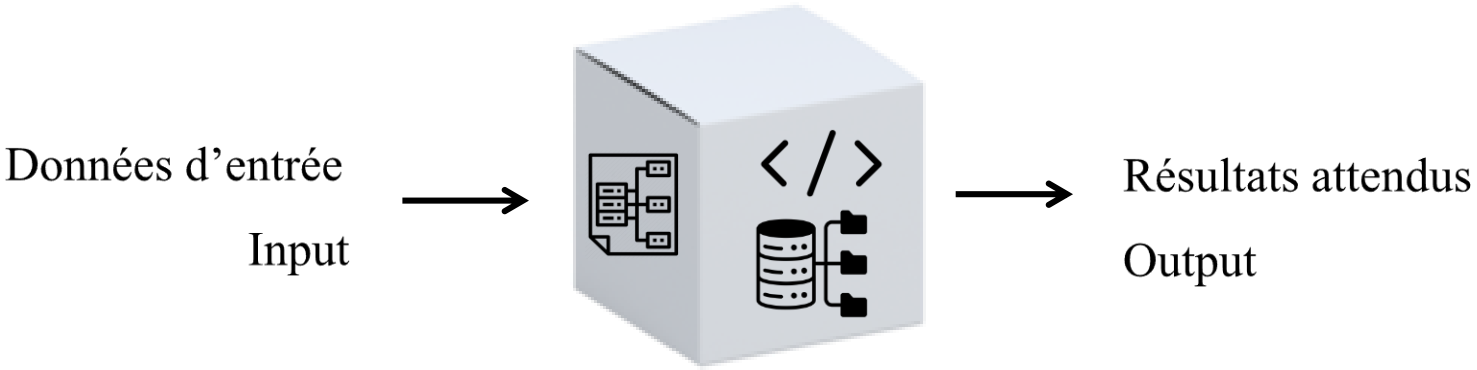
Les techniques de test boîte noire (également connues sous le nom de techniques basées sur les spécifications) sont basées sur une analyse du comportement spécifié de l'objet de test sans référence à sa structure interne.
Les cas de test sont donc indépendants de l'implémentation du logiciel. Par conséquent, si l'implémentation change, mais que le comportement requis reste le même, les cas de test
restent utiles.

Techniques boîte-noire

- Partitions d'équivalence
- Analyse des valeurs limites
- Tests de table de decision
- Tests de transitions d'état
- Tests des cas d'utilisation

TESTS BOÎTE BLANCHE



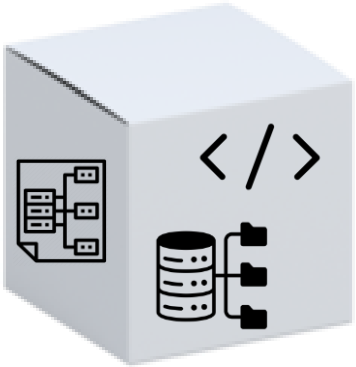


- Le test boîte blanche est basé sur la structure et dérive les tests de l'implémentation ou de la structure interne du système (par exemple, le code, l'architecture, les flux métier et les flux de données).
- L'objectif principal du test boîte blanche est de couvrir la structure sous-jacente par les tests jusqu'au niveau acceptable.
- Les tests boîte-blanche sont des tests basés sur la structure ou l'implémentation interne du système. La structure interne peut comprendre le code, l'architecture, les flux de travail et/ou les flux de données au sein du système.

TESCHNIQUES DE TEST BOÎTE BLANCHE

Tests fonctionnels | Tests non fonctionnels

Test boîte noire | Test boîte blanche



techniques boîte-blanche

Test d'instructions

Test de décisions

EXEMPLES DE TESTS BOÎTE-BLANCHE

Tests fonctionnels | Tests non fonctionnels
Test boîte noire | Test boîte blanche

Types de test et niveaux de test- pour une application bancaire

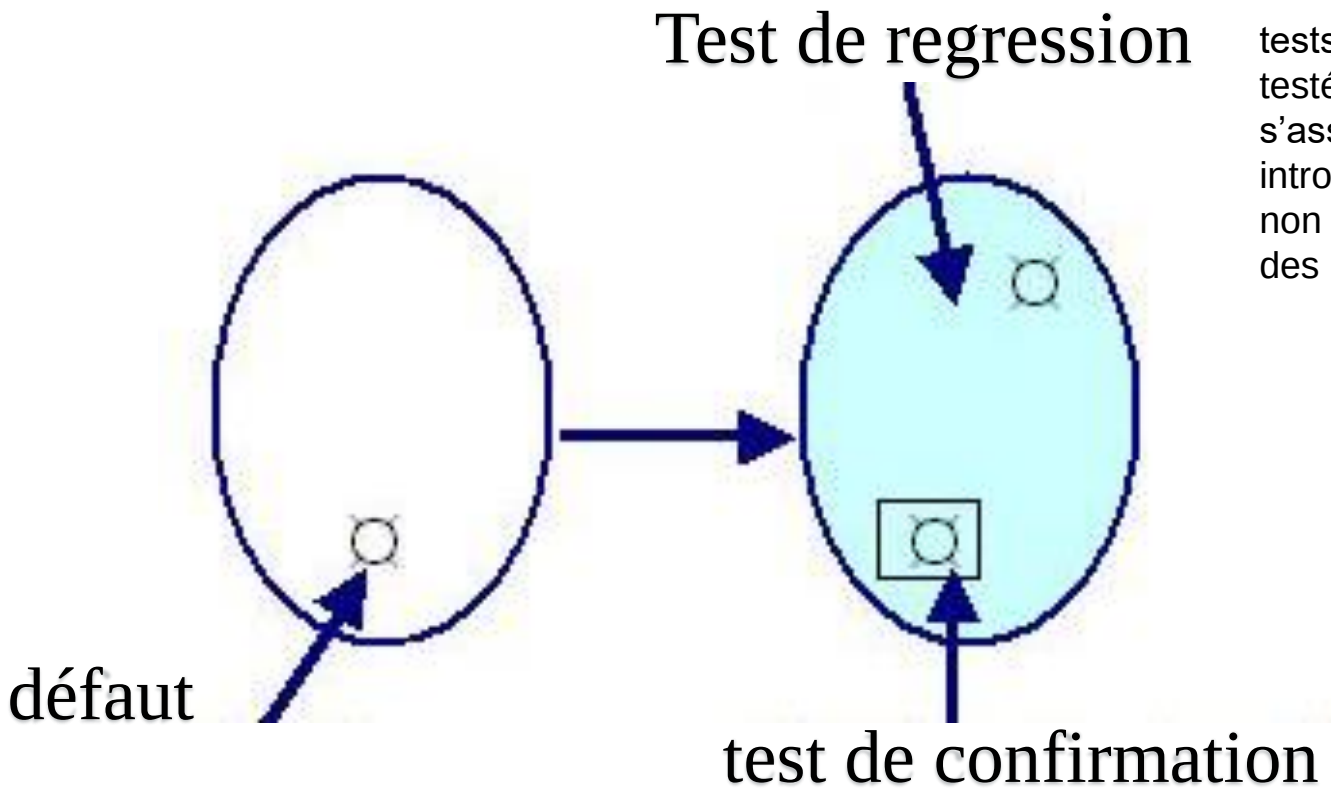
- **Pour les tests de composants**, les tests sont conçus pour obtenir une couverture complète des instructions et des décisions pour tous les composants qui effectuent des calculs financiers
- **Pour les tests d'intégration de composants**, les tests sont conçus pour vérifier comment chaque écran de l'interface du navigateur transmet les données à l'écran suivant et à la couche métier.
- **Pour les tests système**, les tests sont conçus pour couvrir les séquences de pages Web qui peuvent se succéder pendant une demande de ligne de crédit.
- **Pour les tests d'intégration de système**, les tests sont conçus pour exercer tous les types de requête possibles envoyés au micro-service de calcul du taux de crédit.
- **Pour les tests d'acceptation**, les tests sont conçus pour couvrir toutes les structures de fichiers de données financières prises en charge et les plages de valeurs pour les transferts de banque à banque

Tests de régression

Tests de confirmation

LES TESTS LIÉS AUX CHANGEMENTS

Lorsque des **modifications** sont apportées à un système, que ce soit pour corriger un défaut ou en raison **d'une fonctionnalité nouvelle ou modifiée**, des tests devraient être effectués pour confirmer que les modifications ont corrigé le défaut ou implémenté la fonctionnalité correctement, et n'ont pas causé de conséquences préjudiciables inattendues.



tests d'un programme préalablement testé, après une modification, pour s'assurer que des défauts n'ont pas été introduits ou découverts dans des parties non modifiées du logiciel, comme suites des modifications effectuées.

tests dynamiques effectués après la correction de défauts dans le but de confirmer que les défaillances causées par ces défauts ne se produisent plus.

LES TESTS LIÉS AUX CHANGEMENTS

Tests de confirmation

- Après la correction d'un défaut, le logiciel peut être testé avec tous les cas de test qui ont échoué en raison du défaut, qui doivent être ré-exécutés sur la nouvelle version du logiciel.
- Le logiciel peut également être testé avec de nouveaux tests si, par exemple, le défaut était une fonctionnalité manquante.
- A minima, les étapes pour reproduire le(s) défaut(s) causé(s) par le défaut doivent être ré-exécutées sur la nouvelle version du logiciel.
- **Le but d'un test de confirmation est de confirmer que le défaut d'origine a été réparé avec succès.**

Tests de régression

- Il est possible qu'une modification apportée à une partie du code, qu'il s'agisse d'une correction ou d'un autre type de modification, puisse accidentellement affecter le comportement d'autres parties du code, que ce soit au sein du même composant, dans d'autres composants du même système ou même dans d'autres systèmes.
- Les changements peuvent inclure des changements de l'environnement, comme une nouvelle version d'un système d'exploitation ou d'un système de gestion de base de données.
- De tels effets de bord involontaires sont appelés régressions.
- **Les tests de régression sont des tests visant à détecter de tels effets de bord involontaires**

EXEMPLES DE TESTS LIÉS AUX CHANGEMENTS

Types de test et niveaux de test - pour une application bancaire

- **Pour les tests de composants**, des tests de régression automatisés sont construits pour chaque composant et inclus dans le framework d'intégration continue.
- **Pour les tests d'intégration de composants**, les tests sont conçus pour confirmer les corrections des défauts liés à l'interface au fur et à mesure que ces corrections sont intégrées dans le référentiel de code.
- **Pour les tests système**, tous les tests d'un workflow donné sont ré-exécutés si un écran de ce workflow change.
- **Pour les tests d'intégration système**, les tests de l'application interagissant avec le micro-service de calcul du taux de crédit sont ré-exécutés quotidiennement dans le cadre du déploiement continu de ce micro-service.
- **Pour les tests d'acceptation**, tous les tests échoués précédemment sont ré-exécutés après la correction d'un défaut constaté lors des tests d'acceptation.

Des tests de confirmation et de régression sont effectués à tous les niveaux de tests

LES TESTS DE MAINTENANCE

LES TESTS DE MAINTENANCE

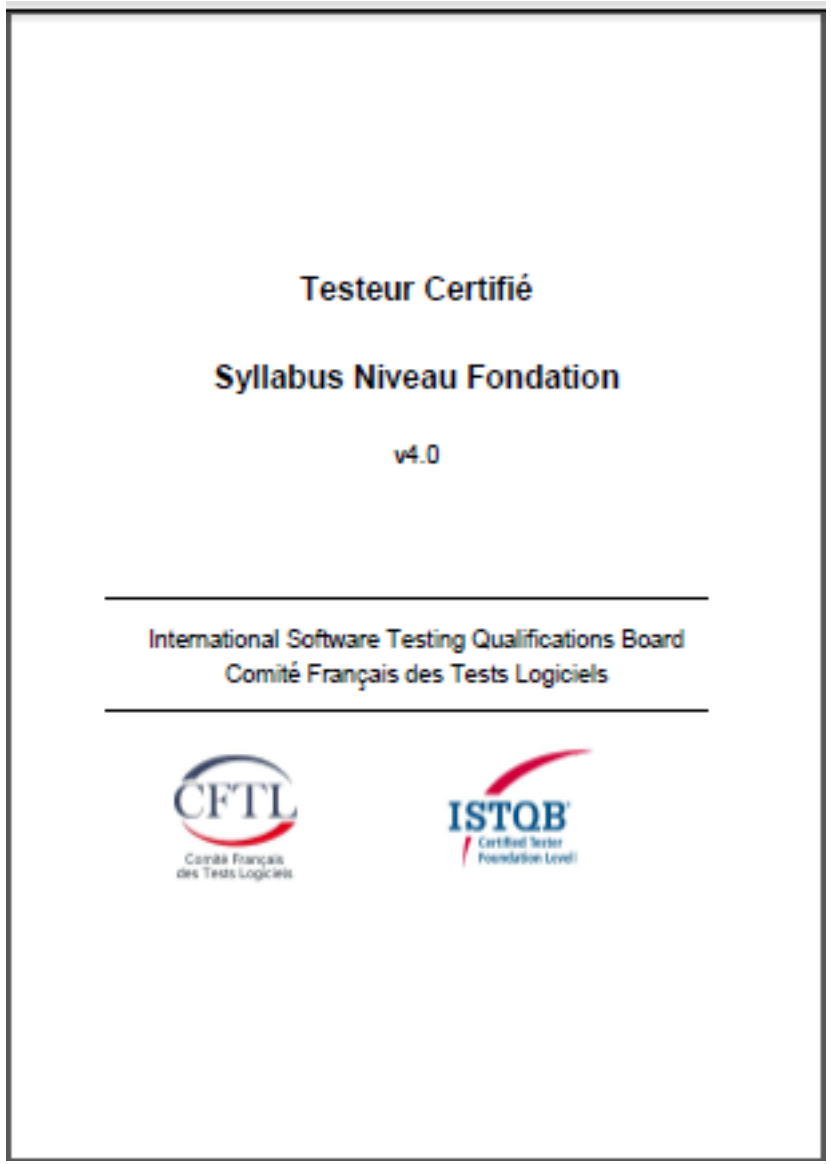
LA MAINTENANCE ?

- Une fois déployée dans les environnements de production, les logiciels et les systèmes doivent être maintenus.
- Des changements de diverses sortes sont presque inévitables dans les logiciels et les systèmes livrés, soit pour corriger des défauts découverts lors de l'utilisation opérationnelle, soit pour ajouter de nouvelles fonctionnalités, soit pour supprimer ou modifier des fonctionnalités déjà livrées.
- La maintenance est également nécessaire pour préserver ou améliorer les caractéristiques de qualité non fonctionnelles du composant ou du système pendant toute sa durée de vie, en particulier la performance, la compatibilité, la fiabilité, la sécurité et la portabilité .

LES TESTS DE MAINTENANCE

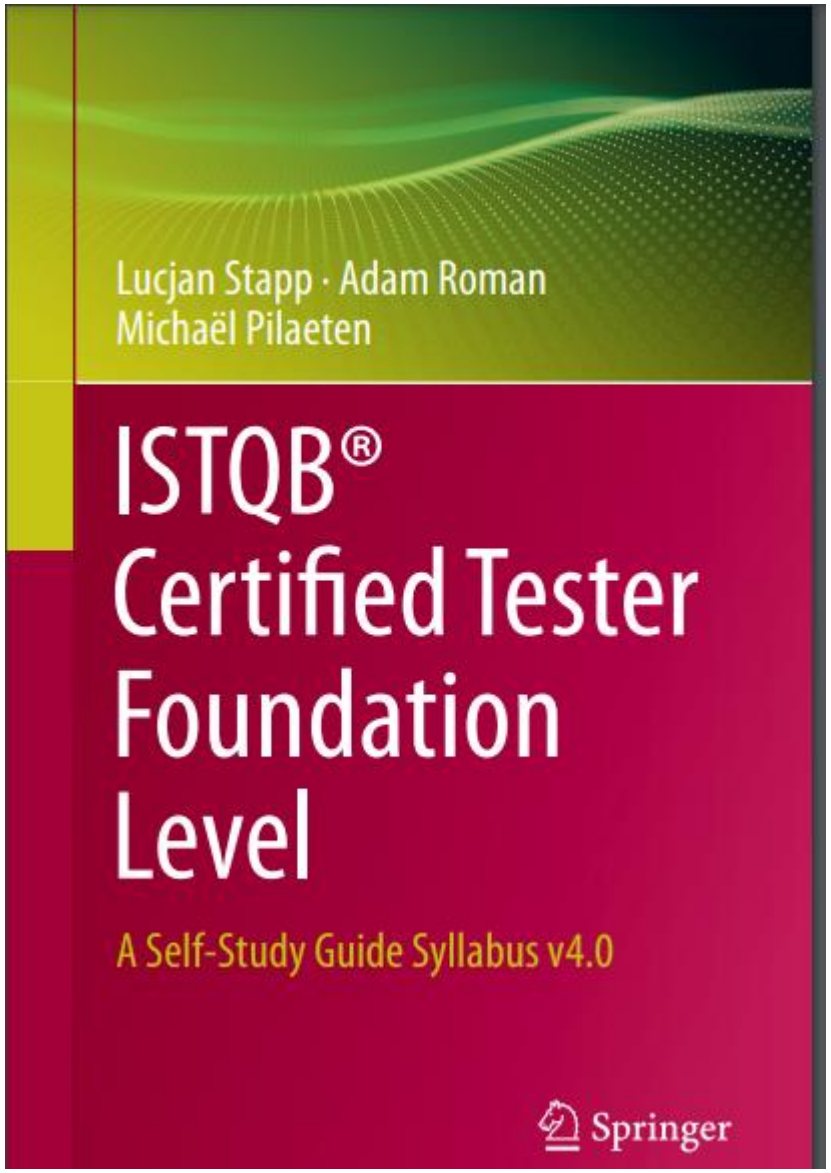
- Une version de maintenance peut nécessiter des tests de maintenance à plusieurs niveaux de test, en utilisant différents types de test, en fonction de sa portée.
- L'étendue des tests de maintenance dépend des éléments suivants :
  - Le degré de risque du changement, par exemple, la mesure dans laquelle le périmètre modifié du logiciel communique avec d'autres composants ou systèmes
  - La taille du système existant
  - La taille du changement

RÉFÉRENCES



Testeur Certifié

Syllabus Niveau Fondation

v4.0

International Software Testing Qualifications Board
Comité Français des Tests Logiciels